

AI Career Path & Tech Stack Recommender

Project Report

An explainable AI recommendation system for suggesting suitable career paths and technology stacks based on user skills, goals, experience level, and interests.

Project Type	AI / Machine Learning / Streamlit Web Application
Main Technique	Content-based filtering using TF-IDF and cosine similarity
Core Output	Career recommendations, match percentage, skill gaps, tools, and learning roadmap
Prepared By	Sanjula Supunthaka

1. Executive Summary

This report documents the AI Career Path & Tech Stack Recommender project. The project helps students and beginners identify suitable technology careers by comparing their skills, career goal, experience level, and interests against a structured dataset of career roles. The application is built with Python and Streamlit, while the recommendation engine uses TF-IDF vectorization, cosine similarity, hybrid scoring, and explainable skill-gap analysis.

The strongest part of the project is not only giving a recommendation, but explaining why the recommendation was made. This makes the system more useful for learning guidance, internship preparation, and portfolio demonstration.

2. Problem Statement

Many students know a few technical skills such as Python, SQL, or HTML, but they are unsure which career path fits those skills. Basic advice such as “learn AI” or “become a developer” is too broad and does not identify missing skills or next learning steps. A recommendation system can reduce this confusion by mapping a learner profile to career roles and showing clear improvement areas.

3. Project Objectives

- Recommend suitable career paths based on user-entered skills, career goal, level, and interests.
- Calculate a match percentage using a weighted hybrid scoring method.
- Show matched skills and missing skills for each recommended role.
- Suggest the relevant technology stack and a personalized learning roadmap.
- Provide a simple Streamlit user interface that can be run locally and demonstrated easily.

4. System Overview

The system follows a content-based recommendation approach. Each career role in the dataset contains required skills, career goal keywords, difficulty level, tools, and roadmap. The user profile is converted into text features and compared against each role profile.

Input	Processing	Output
Skills, career goal, level, interests	Text cleaning, TF-IDF vectorization, cosine similarity, hybrid score calculation, ranking	Top career roles, match percentage, matched skills, missing skills, tools, roadmap

5. Dataset Description

The project uses a CSV file named `career_dataset.csv`. It contains 21 career role records and 6 columns. Each row represents one career path profile.

Column	Purpose
role	Name of the career path, such as Data Scientist or AI Engineer.
required_skills	Skills required for the role.
career_goal	Goal keywords related to the role.
difficulty	Role level: beginner, intermediate, or advanced.
tools	Suggested tools and technologies.

Role	Difficulty	Tools / Technologies
Data Scientist	intermediate	Python, SQL, Pandas, NumPy, Scikit-learn, Matplotlib
Machine Learning Engineer	advanced	Python, TensorFlow, PyTorch, Docker, FastAPI, Git
AI Engineer	advanced	Python, PyTorch, TensorFlow, Hugging Face, OpenCV
Data Analyst	beginner	Excel, SQL, Power BI, Tableau, Python
Business Intelligence Analyst	intermediate	Power BI, Tableau, SQL, Excel
Backend Developer	intermediate	Python, Django, Flask, Node.js, SQL, Git
Frontend Developer	beginner	HTML, CSS, JavaScript, React, Git

6. Methodology

Step 1 - User Input: The user enters skills, career goal, current level, and interests through the Streamlit interface.

Step 2 - Text Cleaning: The `clean_text` function converts text to lowercase, removes unnecessary symbols, and normalizes spacing.

Step 3 - Vectorization: TF-IDF converts skill and goal text into numerical vectors so that similarity can be calculated.

Step 4 - Similarity Calculation: Cosine similarity measures how close the user skills and goal are to each role profile.

Step 5 - Hybrid Scoring: Skill similarity, goal similarity, difficulty alignment, and interest overlap are combined using adjustable weights.

Step 6 - Explanation Layer: The system displays matched skills, missing skills, tools, and roadmap for each recommendation.

7. Hybrid Score Formula

The default final score uses the following weighted formula:

Final Score = 65% Skill Similarity + 20% Goal Similarity + 10% Difficulty Alignment + 5% Interest Match

In the Streamlit sidebar, these weights can be adjusted. The application normalizes the total weight automatically, which is a good design choice because it prevents invalid scoring when the total is not exactly 1.0.

8. Main Files and Their Responsibilities

File	Responsibility
<code>app.py</code>	Creates the Streamlit user interface, collects user input, displays charts, metrics, recommendations, and dataset preview.
<code>recommender.py</code>	Contains the recommendation logic, text preprocessing, similarity calculation, difficulty alignment, skill-gap analysis, and final ranking.
<code>career_dataset.csv</code>	Stores the career role profiles used by the recommender.
<code>requirements.txt</code>	Lists the Python packages needed to run the project.
<code>README.md</code>	Explains the project overview, methodology, run commands, and future improvements.

9. Key Code Logic

The recommender class loads the dataset, validates required columns, builds TF-IDF matrices, and produces ranked recommendations. The most important logic is the weighted score calculation.

```
final_scores = (
    weights["skill"] * skill_scores +
    weights["goal"] * goal_scores +
    weights["difficulty"] * difficulty_scores +
    weights["interest"] * interest_scores
)
```

This formula is understandable and easy to explain in a viva or internship review. However, it is still a rule-based scoring model, not a trained supervised model. That distinction should be stated clearly when presenting the project.

10. User Interface Features

- Sidebar controls for number of recommendations and scoring weights.
- Text input area for user skills and interests.
- Career goal and level selection boxes.
- Recommendation button to run the engine.
- Bar chart for match percentages.
- Cards showing each recommendation with metrics and explanation.
- Detailed score table and dataset preview.

11. Sample Test Case and Output

Sample input used for testing:

Field		Value		
Skills		Python, SQL, Machine Learning		
Career Goal		Artificial Intelligence		
Level		Intermediate		
Interests		AI projects, real-world applications		
Recommended Role	Match	Level	Matched Skills	Missing Skills
Data Scientist	46.88%	intermediate	python, sql, machine learning	statistics, pandas, numpy, data visualization, scikit-learn
Machine Learning Engineer	31.86%	advanced	python, machine learning	deep learning, tensorflow, pytorch, model deployment, mlops
Research Assistant in AI	30.81%	advanced	python, machine learning	deep learning, research writing, literature review, statistics, experiments
AI Engineer	20.08%	advanced	python	deep learning, nlp, computer vision, transformers, tensorflow
Full Stack Developer	19.86%	intermediate	python, sql	html, css, javascript, react, nodejs

The result correctly ranks Data Scientist highest for the sample input because the user skills directly include Python, SQL, and Machine Learning, which strongly match the Data Scientist role profile.

12. How to Run the Project

Open the project folder in VS Code or a terminal, then run the following commands:

```
pip install -r requirements.txt  
streamlit run app.py
```

After running the command, Streamlit will provide a local browser URL such as <http://localhost:8501>.

13. Strengths of the Project

- The system is explainable, not just a black-box recommendation output.
- The hybrid score combines several factors instead of using skills alone.
- The Streamlit interface makes the project easy to demonstrate.
- The dataset structure is simple and can be expanded easily.
- The roadmap output turns the recommendation into practical learning guidance.

14. Limitations and Critical Review

- The dataset is small, so recommendations are limited by the available role profiles.
- The scoring is manually weighted; it is not learned from real user outcome data.
- Skill matching depends on text similarity, so synonyms may not always match correctly.
- The project does not yet verify whether a roadmap is realistic for a specific time period.
- There is no login, saved profile, or feedback loop to improve future recommendations.

For a stronger presentation, describe this as an explainable content-based recommender, not as a fully intelligent career advisor. That wording is more accurate and more defensible.

15. Future Improvements

- Increase the dataset size with more career roles and skill variations.
- Add course recommendation links for each missing skill.
- Use sentence embeddings to improve semantic matching beyond keyword similarity.
- Add resume parsing to extract skills automatically from a PDF resume.
- Deploy the application using Streamlit Cloud.
- Collect feedback from users and tune recommendation weights using real data.

16. Conclusion

The AI Career Path & Tech Stack Recommender successfully demonstrates how AI and machine learning concepts can be used to guide students toward suitable technology careers. It uses a clear pipeline: user input, text preprocessing, TF-IDF vectorization, cosine similarity, hybrid scoring, ranking, and explanation. The project is suitable for an internship portfolio because it combines practical Python development, machine learning logic, Streamlit interface design, and explainable output.

Appendix A - Project Structure

AI-Career-Tech-Stack-Recommender/

```
|— app.py  
|— recommender.py  
|— career_dataset.csv  
|— requirements.txt  
|— README.md
```

Appendix B - Dependencies

- streamlit
- pandas
- scikit-learn
- numpy